

subject	title	check ☐ ☐ ☐ ☐ ☐
김태형교수님	김태형교수님 - Node.js Express 개념정리	date

2026.05.14

Express 개념정리

express()	서버 앱 객체 생성
-----------	------------

- app.set('view engine', 'ejs'): EJS 템플릿 엔진 설정
- app.use(express.urlencoded(...)): form POST 데이터를 req.body로 받기 위한 미들웨어

app.get	GET 요청 라우트
app.post	POST 요청 라우트
res.render	템플릿 파일을 HTML로 렌더링
res.redirect	처리 후 다른 경로로 이동
app.listen	포트에서 서버 실행

subject	title	check ☐ ☐ ☐ ☐ ☐
김태형교수님	김태형교수님 - Node.js Express 개념정리	date

2026.06.05

Express 라우트 순서 문제

핵심

Express는 위에서 아래로 라우트를 검사한다. 그래서 구체적인 경로(`/posts/form`)보다 동적 경로(`/posts/:postId`)가 먼저 있으면 `/posts/form` 요청도 `/posts/:postId`에 걸린다.

문제 원인

```
app.get('/posts/:postId', ...)
app.get('/posts/form', ...)
```

위 명령어/코드는 수업 중 사용한 실행 흐름을 그대로 따라가기 위한 예시.

주석

이 순서에서는 `/posts/form`으로 접속해도 Express가 먼저 `/posts/:postId`와 매칭한다. 그 결과 `postId` = "form"이 되고, DB에서 `post\_id`가 "form"인 글을 찾으려고 해서 게시글을 못 찾는다.

주의

라우트에서 `:postId` 같은 파라미터는 거의 모든 문자열을 받을 수 있으므로, 고정 경로보다 아래에 두는 것이 안전하다.

subject	title	check ☐ ☐ ☐ ☐ ☐
김태형교수님	김태형교수님 - Node.js Express 개념정리	date

수정 코드: 고정 경로를 먼저 선언

```
app.get('/posts/form', function(req, res){
  res.render('form');
});

app.get('/posts/:postId', function(req, res){
  const postId = req.params.postId;

  db.query("SELECT * FROM posts WHERE post_id = ?", [postId], function(err, posts){
    if (err) {
      console.log(err);
      return res.send("DB 오류");
    }

    if (posts.length === 0) {
      return res.send("게시글 없음");
    }

    res.render('post', { post: posts[0] });
  });
});
```

위 명령어/코드는 수업 중 사용한 실행 흐름을 그대로 따라가기 위한 예시.

주석 `/posts/form`처럼 정확히 정해진 주소를 먼저 등록하고, `/posts/:postId`처럼 값이 변하는 동적 주소를 나중에 등록한다.

주석 `req.params.postId`는 URL의 `:postId` 부분에 들어온 값을 가져온다. `/posts/3`이면 `postId`는 `"3"`이고, 잘못된 순서에서는 `/posts/form`도 `postId`가 `"form"`이 된다.

주석 SQL의 `?`와 `[postId]`는 prepared statement 방식이다. 직접 문자열을 이어 붙이지 않고 값을 따로 넘겨 SQL injection 위험을 줄인다.

주석 `posts.length === 0`이면 조회 결과가 없는 것이므로 `게시글 없음`을 반환한다. 결과가 있으면 첫 번째 행인 `posts[0]`을 `post.ejs`에 넘긴다.

subject	title	check ☐ ☐ ☐ ☐ ☐
김태형교수님	김태형교수님 - Node.js Express 개념정리	date

post.ejs 출력 코드

```
<h5><%= post.title %></h5>
<pre class="content"><%= post.content %></pre>
```

위 명령어/코드는 수업 중 사용한 실행 흐름을 그대로 따라가기 위한 예시.

주석 `<%= post.title %>`은 서버에서 넘긴 `post` 객체의 제목을 출력한다. `<%= post.content %>`는 본문을 출력한다.

주석 `

```
` 태그는 줄바꿈과 공백을 비교적 그대로 보여주기 때문에 게시글 본문 출력에 사용할 수 있다.
```

정리 Express 라우트는 작성 순서가 중요하다. `/posts/form` 같은 고정 라우트는 `/posts/:postId` 같은 동적 라우트보다 위에 뒤야 한다.

라우트 순서 개념: 넓은 조건이 먼저 잡아먹는 문제

핵심 이 문제는 단순히 "라우터 문제"라기보다, 넓은 조건을 먼저 쓰면 구체적인 조건까지 먼저 잡아먹는 문제다. Express 라우트도 `if / else if`처럼 위에서 아래로 먼저 맞는 조건을 찾는다.

C 언어 if문으로 비유

```
int score = 95;

if (score >= 0) {
  printf("점수 있음");
} else if (score >= 90) {
  printf("A");
}
```

위 명령어/코드는 수업 중 사용한 실행 흐름을 그대로 따라가기 위한 예시.

주석 `95`는 `score >= 90`에도 맞지만, 위에 있는 더 넓은 조건 `score >= 0`에 먼저 걸린다. 그래서 `A`까지 가지 못한다.

```
int score = 95;

if (score >= 90) {
  printf("A");
} else if (score >= 0) {
  printf("점수 있음");
}
```

위 명령어/코드는 수업 중 사용한 실행 흐름을 그대로 따라가기 위한 예시.

주석 더 구체적인 조건인 `score >= 90`을 먼저 검사해야 원하는 결과가 나온다.

subject	title	check ☐ ☐ ☐ ☐ ☐
김태형교수님	김태형교수님 - Node.js Express 개념정리	date

Python 조건문으로 비유

```
path = "/posts/form"

if path.startswith("/posts/"):
    print("게시글 상세")
elif path == "/posts/form":
    print("글쓰기 폼")
```

위 명령어/코드는 수업 중 사용한 실행 흐름을 그대로 따라가기 위한 예시.

주석 `/posts/form`은 `path.startswith("/posts/")`에도 맞기 때문에 아래의 `path == "/posts/form"`까지 가지 못한다.

```
path = "/posts/form"

if path == "/posts/form":
    print("글쓰기 폼")
elif path.startswith("/posts/"):
    print("게시글 상세")
```

위 명령어/코드는 수업 중 사용한 실행 흐름을 그대로 따라가기 위한 예시.

주석 고정된 정확한 조건을 먼저 두고, 넓은 조건을 나중에 뒤야 한다.

Express에서 실제로 생긴 문제

```
app.get('/posts/:postId', ...)
app.get('/posts/form', ...)
```

위 명령어/코드는 수업 중 사용한 실행 흐름을 그대로 따라가기 위한 예시.

주석 `/posts/:postId`는 `/posts/` 뒤에 오는 값을 전부 `postId`로 받을 수 있는 넓은 조건이다. 그래서 `/posts/form`으로 들어가도 먼저 `/posts/:postId`에 걸린다.

```
req.params.postId = "form";
```

위 명령어/코드는 수업 중 사용한 실행 흐름을 그대로 따라가기 위한 예시.

주석 Express가 `postId`를 모르는 것이 아니라, `postId`를 `"form"`이라고 잘못 알아듣는 것이 문제다.

```
SELECT * FROM posts WHERE post_id = 'form';
```

위 명령어/코드는 수업 중 사용한 실행 흐름을 그대로 따라가기 위한 예시.

주석 `post_id`는 보통 숫자인데 `"form"`으로 조회하니 게시글을 찾지 못한다. 그 결과 `posts[0]`이 `undefined`가 되고, `post.ejs`에서 `post.title`을 출력할 때 문제가 생길 수 있다.

subject	title	check ☐ ☐ ☐ ☐ ☐
김태형교수님	김태형교수님 - Node.js Express 개념정리	date

올바른 라우트 순서

```
app.get('/posts/form', function(req, res){
  res.render('form');
});

app.get('/posts/:postId', function(req, res){
  const postId = req.params.postId;
  // 게시물 상세 조회
});
```

위 명령어/코드는 수업 중 사용한 실행 흐름을 그대로 따라가기 위한 예시.

주석 `/posts/form`처럼 정확히 정해진 고정 라우트를 먼저 두고, `/posts/:postId`처럼 넓게 받는 동적 라우트를 나중에 둔다.

정리 `if / else if`, Python 조건문, Express 라우트는 모두 위에서 아래로 먼저 맞는 조건을 처리한다. 그래서 좁고 구체적인 조건을 먼저, 넓고 일반적인 조건을 나중에 뒤야 한다.

subject	title	check ☐ ☐ ☐ ☐ ☐
김태형교수님	김태형교수님 - Node.js Express 개념정리	date

2026.06.26

Express 미들웨어 흐름: Spring Filter처럼 이해하기

핵심

그림의 흐름은 Spring의 Filter/Interceptor처럼 요청이 라우터에 도착하기 전에 여러 미들웨어를 순서대로 통과하는 구조다. Express에서는 `app.use(...)`로 등록한 미들웨어가 위에서 아래로 실행되고, 각 미들웨어는 필요한 정보를 `req`, `res`, `res.locals`에 붙인 뒤 `next()`로 다음 단계에 넘긴다.

출처: Express 공식 문서는 Express 앱을 "middleware function calls의 series"로 설명하고, 미들웨어가 `req`, `res`, `next`에 접근하며 요청-응답 흐름을 끝내거나 다음 미들웨어로 넘길 수 있다고 설명한다. 참고:

<https://expressjs.com/en/guide/using-middleware/>

전체 흐름

요청

- > 폼데이터 파싱
- > 쿠키 파싱
- > 세션 인식
- > 유저정보 세팅(res.locals.user)
- > app.get / app.post 라우트 핸들러
- > ejs 렌더링
- > 완성된 HTML 응답

위 명령어/코드는 수업 중 사용한 실행 흐름을 그대로 따라가기 위한 예시.

주석

라우터가 바로 실행되는 것이 아니라, 라우터 앞에 등록된 공통 처리들이 먼저 실행된다. 그래서 로그인 여부, 세션 사용자, 폼 입력값 같은 정보를 라우트에서 바로 사용할 수 있다.

1단계: 요청(Request)

브라우저가 서버에 요청을 보낸다. 예를 들어 로그인 폼 제출, 게시글 작성, 게시글 목록 조회가 모두 요청이다.

POST /login

Content-Type: application/x-www-form-urlencoded

Cookie: connect.sid=...

위 명령어/코드는 수업 중 사용한 실행 흐름을 그대로 따라가기 위한 예시.

주석

요청에는 URL, HTTP method, header, cookie, body 같은 정보가 들어 있다. Express는 이 요청을 `req` 객체로 다룬다.

subject	title	check ☐ ☐ ☐ ☐ ☐
김태형교수님	김태형교수님 - Node.js Express 개념정리	date

2단계: 폼데이터 파싱

HTML form에서 전송된 데이터를 `req.body`로 쓰려면 먼저 파싱 미들웨어가 필요하다.

```
app.use(express.urlencoded({ extended: true }));
```

위 명령어/코드는 수업 중 사용한 실행 흐름을 그대로 따라가기 위한 예시.

주석

`express.urlencoded()`는 URL-encoded 형식의 요청 body를 파싱한다. Express 공식 문서에서도 `express.urlencoded`를 URL-encoded payload를 파싱하는 built-in middleware로 설명한다.

참고: <https://expressjs.com/en/guide/using-middleware/>

```
<form method="post" action="/login">
  <input name="email">
  <input name="password">
</form>
```

위 명령어/코드는 수업 중 사용한 실행 흐름을 그대로 따라가기 위한 예시.

```
app.post('/login', function(req, res){
  console.log(req.body.email);
  console.log(req.body.password);
});
```

위 명령어/코드는 수업 중 사용한 실행 흐름을 그대로 따라가기 위한 예시.

주의

`app.use(express.urlencoded(...))`가 `app.post('/login', ...)`보다 아래에 있으면 라우트가 먼저 실행되기 때문에 `req.body`를 제대로 못 쓸 수 있다.

subject	title	check ☐ ☐ ☐ ☐ ☐
김태형교수님	김태형교수님 - Node.js Express 개념정리	date

3단계: 쿠키 파싱

쿠키는 브라우저가 매 요청마다 서버로 보내는 작은 데이터다. 로그인 세션 ID도 보통 쿠키에 담긴다.

```
const cookieParser = require('cookie-parser');
app.use(cookieParser());
```

위 명령어/코드는 수업 중 사용한 실행 흐름을 그대로 따라가기 위한 예시.

주석

`cookie-parser`는 요청의 `Cookie` header를 파싱해서 `req.cookies` 객체로 만들어 준다. 공식 문서도 Cookie header를 파싱해 `req.cookies`에 담는다고 설명한다.

참고: <https://expressjs.com/en/resources/middleware/cookie-parser/>

```
app.get('/debug-cookie', function(req, res){
  console.log(req.cookies);
  res.send('cookie check');
});
```

위 명령어/코드는 수업 중 사용한 실행 흐름을 그대로 따라가기 위한 예시.

주의

쿠키는 클라이언트가 들고 있는 값이므로 중요한 정보를 그대로 저장하면 안 된다. 보통은 세션 ID 같은 식별자만 넣고, 실제 로그인 정보는 서버 쪽에 둔다.

subject	title	check ☐ ☐ ☐ ☐ ☐
김태형교수님	김태형교수님 - Node.js Express 개념정리	date

4단계: 세션 인식

세션은 "이 요청을 보낸 사용자가 누구인지" 서버가 기억하기 위한 장치다. 브라우저는 쿠키로 세션 ID를 보내고, 서버는 그 ID를 이용해 서버 쪽 저장소에서 세션 데이터를 찾는다.

```
const session = require('express-session');

app.use(session({
  secret: 'keyboard cat',
  resave: false,
  saveUninitialized: false
}));
```

위 명령어/코드는 수업 중 사용한 실행 흐름을 그대로 따라가기 위한 예시.

주석

`express-session` 공식 문서는 세션 데이터가 쿠키 자체에 저장되는 것이 아니라, 쿠키에는 세션 ID만 저장되고 실제 세션 데이터는 서버 쪽에 저장된다고 설명한다.

참고: <https://expressjs.com/en/resources/middleware/session/>

```
app.post('/login', function(req, res){
  req.session.userId = 3;
  res.redirect('/');
});
```

위 명령어/코드는 수업 중 사용한 실행 흐름을 그대로 따라가기 위한 예시.

주석

로그인 성공 후 `req.session.userId`에 사용자 ID를 넣어 두면, 다음 요청부터 세션을 통해 "로그인한 사용자"를 알아볼 수 있다.

subject	title	check ☐ ☐ ☐ ☐ ☐
김태형교수님	김태형교수님 - Node.js Express 개념정리	date

5단계: 유저정보 세팅(res.locals.user)

세션에는 보통 `userId`만 들어 있다. EJS 화면에서 사용자 닉네임, 로그인 여부를 편하게 쓰려면 DB에서 사용자 정보를 가져와 `res.locals.user`에 넣어 둔다.

```
app.use(function(req, res, next){
  res.locals.user = null;

  if (!req.session.userId) {
    return next();
  }

  db.query(
    'SELECT user_id, nickname, email FROM users WHERE user_id = ?',
    [req.session.userId],
    function(err, users){
      if (err) {
        return next(err);
      }

      res.locals.user = users[0] || null;
      next();
    }
  );
});
```

위 명령어/코드는 수업 중 사용한 실행 흐름을 그대로 따라가기 위한 예시.

주석

`res.locals`는 이번 요청-응답 사이클에서만 쓰는 응답 지역 변수다. 여기에 `user`를 넣으면 이후 라우터와 EJS에서 `user`를 공통으로 사용할 수 있다.

주의

`res.locals.user`를 세팅하는 미들웨어는 EJS를 렌더링하는 라우트보다 위에 있어야 한다. 그래야 모든 화면에서 로그인 사용자 정보를 사용할 수 있다.

subject	title	check ☐ ☐ ☐ ☐ ☐
김태형교수님	김태형교수님 - Node.js Express 개념정리	date

6단계: app.get / app.post 라우트 핸들러

전처리가 끝나면 실제 라우트 핸들러가 실행된다.

```
app.get('/', function(req, res){
  db.query('SELECT * FROM posts ORDER BY post_id DESC', function(err, posts){
    res.render('index', { posts });
  });
});

app.post('/posts', function(req, res){
  const { title, content } = req.body;

  db.query(
    'INSERT INTO posts (user_id, title, content) VALUES (?, ?, ?)',
    [req.session.userId, title, content],
    function(err){
      res.redirect('/');
    }
  );
});
```

위 명령어/코드는 수업 중 사용한 실행 흐름을 그대로 따라가기 위한 예시.

주석 이 시점에는 앞 단계에서 만든 `req.body`, `req.cookies`, `req.session`, `res.locals.user`를 사용할 수 있다.

7단계: EJS 렌더링

라우트 핸들러가 `res.render()`를 호출하면 EJS 파일에 데이터가 들어가고 HTML이 만들어진다.

```
res.render('index', { posts });
```

위 명령어/코드는 수업 중 사용한 실행 흐름을 그대로 따라가기 위한 예시.

```
<% if (user) { %>
<p><%= user.nickname %>님 로그인 중</p>
<% } else { %>
<a href="/login">로그인</a>
<% } %>
```

위 명령어/코드는 수업 중 사용한 실행 흐름을 그대로 따라가기 위한 예시.

주석 Express 공식 문서는 템플릿 엔진이 템플릿 파일의 변수를 실제 값으로 바꾸고, 클라이언트로 보낼 HTML로 변환한다고 설명한다. `res.render()`는 그 템플릿 렌더링을 호출한다.

참고: <https://expressjs.com/en/guide/using-template-engines/>

subject	title	check ☐ ☐ ☐ ☐ ☐
김태형교수님	김태형교수님 - Node.js Express 개념정리	date

8단계: 완성된 HTML 응답

마지막으로 Express는 렌더링된 HTML을 브라우저에 응답한다.

```

서버: index.ejs + posts + user
-> HTML 생성
-> 브라우저로 응답

```

위 명령어/코드는 수업 중 사용한 실행 흐름을 그대로 따라가기 위한 예시.

주석 브라우저는 EJS를 직접 받는 것이 아니라, EJS가 서버에서 실행된 결과인 HTML을 받는다.

순서가 중요한 이유

```

app.use(express.urlencoded({ extended: true }));
app.use(cookieParser());
app.use(session(...));
app.use(loadUserToLocals);

app.get('/', routeHandler);
app.post('/posts', routeHandler);

```

위 명령어/코드는 수업 중 사용한 실행 흐름을 그대로 따라가기 위한 예시.

주석 미들웨어는 위에서 아래로 실행된다. 따라서 데이터를 만들어 주는 미들웨어가 먼저 오고, 그 데이터를 사용하는 라우트가 나중에 와야 한다.

```

express.urlencoded -> req.body 생성
cookieParser -> req.cookies 생성
session -> req.session 생성
loadUserToLocals -> res.locals.user 생성
route handler -> 실제 기능 처리
res.render -> EJS로 HTML 생성

```

위 명령어/코드는 수업 중 사용한 실행 흐름을 그대로 따라가기 위한 예시.

정리 Spring Filter처럼 Express 미들웨어는 요청이 라우터에 들어가기 전 공통 작업을 처리하는 단계다. 폼데이터, 쿠키, 세션, 로그인 사용자 정보를 먼저 준비해 두면 각 `app.get`, `app.post` 라우트에서는 핵심 기능만 작성할 수 있다.